Plan: Computer Vision tránh vật cản cho Drone (mô phỏng trước)

Nov 3, 2026 · @non

Tổng quan

Mục tiêu cuối: một drone trong mô phỏng tự bay từ điểm A tới điểm B, dùng camera để phát hiện và né vật cản, thành công ít nhất 90% trên 50 world random do script tự chấm. Làm hoàn toàn trên máy tính trước, chỉ mua phần cứng khi bản mô phỏng đã chạy ổn.

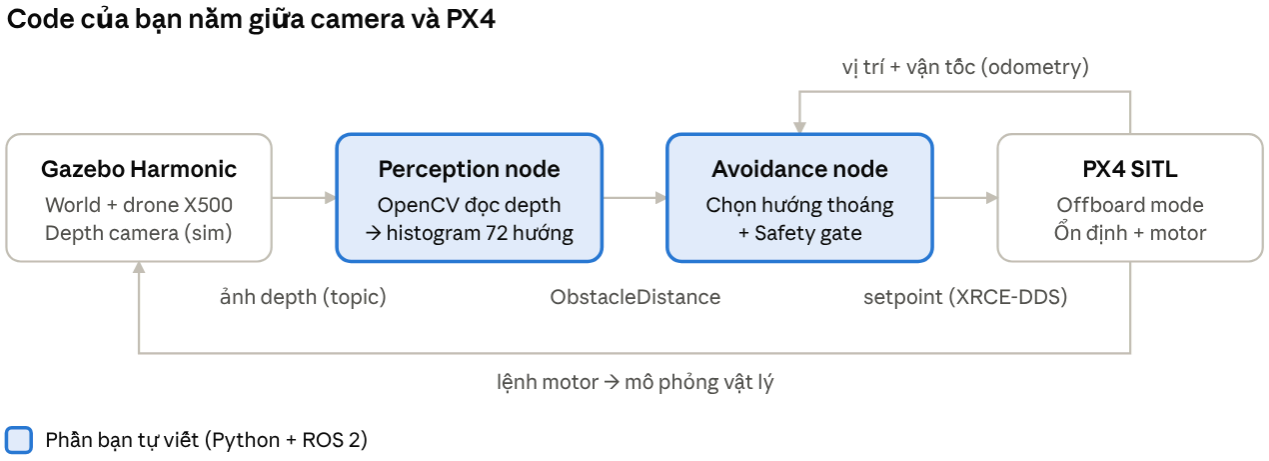
Giả định: tự học, không deadline, mới bắt đầu với computer vision, chưa có drone. Nhịp gợi ý khoảng 8–10 giờ/tuần, tổng cộng tầm 16–20 tuần cho bản lõi.

Nguyên tắc chính:

- Đi từ dễ tới khó: dùng depth camera giả lập trước (có sẵn khoảng cách), sau đó mới chuyển sang camera thường + model ước lượng độ sâu.
- Mỗi giai đoạn phải ra được một thứ chạy được và đo được, không học lý thuyết suông.
- Không train hay fine-tune model nào. Giai đoạn 0–3 không dùng ML; giai đoạn 4 chỉ chạy model pretrained (inference), máy yếu thì thay bằng stereo SGBM chạy CPU.

Ngoài phạm vi (giai đoạn đầu): bay ngoài trời thật, SLAM đầy đủ, nhiều drone, học tăng cường.

Kiến trúc hệ thống

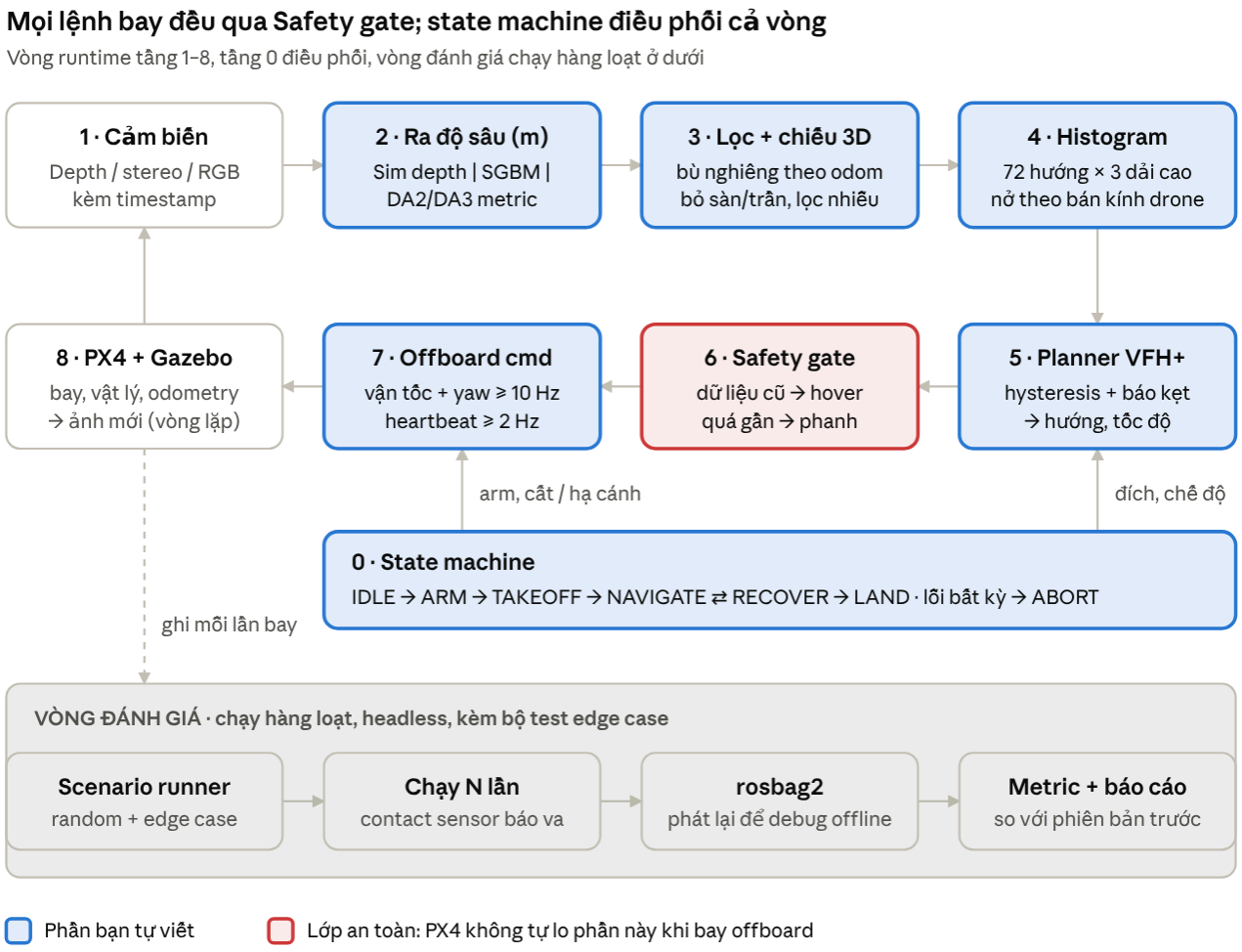


kiến trúc · 4 khối, 1 vòng phản hồi

Bạn chỉ cần viết 2 node ở giữa; Gazebo và PX4 có sẵn. Giai đoạn 4 chỉ thay ô Gazebo bằng camera RGB + Depth Anything, phần còn lại giữ nguyên.

Pipeline hoàn chỉnh

Pipeline có 8 tầng runtime cộng tầng 0 (state machine) điều phối, mỗi tầng test được riêng, và một vòng đánh giá tự động bọc bên ngoài. Thay đổi lớn nhất so với bản cũ: thêm Safety gate, dùng histogram 72 hướng thay cho 3 vùng, và đo mọi thứ bằng script chứ không đếm tay.



pipeline · 8 tầng runtime + vòng đánh giá

Tầng 2 là chỗ duy nhất thay đổi giữa các giai đoạn (sim depth → stereo → model); mọi tầng sau đều nhận cùng một ảnh depth tính bằng mét.

Spec từng tầng

Các con số là giá trị khởi đầu để tune, không phải chuẩn.

Tầng	Input → Output	Message (gợi ý)	Tham số khởi đầu	Test băng
0 · State machine	Trạng thái PX4 + health các node → chế độ + đích	`VehicleStatus` (`nav_state`, arming)	IDLE → ARM → TAKEOFF → NAVIGATE ⇄ RECOVER → LAND; lỗi bất kỳ → ABORT	Mỗi chuyển trạng thái có 1 test; nhóm C trong Edge cases
1 · Cảm biến	Gazebo → ảnh + CameraInfo	`sensor_msgs/Image`, `CameraInfo` qua ros_gz_bridge	Model `gz_x500_depth` có sẵn depth camera kiểu OAK-D (PX4 docs); 320×240 nếu máy yếu	rqt_image_view, kiểm tra tần số và timestamp
2 · Ra độ sâu	Ảnh → depth float32 (m)	`Image` 32FC1	Tầm 0.3–10 m; tách "không hợp lệ vì quá gần" với "vì quá xa"	So với sim depth: sai số AbsRel, ms/frame
3 · Lọc + chiếu 3D	Depth + odometry → điểm 3D trong hệ thế giới đã lọc	Nội bộ node, TF camera → body → world	Bù pitch/roll theo odometry cùng timestamp; bỏ điểm lệch độ cao drone > 0.5 m; percentile 5% mỗi cột; median 3 frame	Tăng tốc gắp trên sàn phẳng: 0 vật cản giả
4 · Histogram	Điểm 3D → 72 khoảng cách × 3 dải cao	`px4_msgs/ObstacleDistance` (dải giữa)	72 bin × 5°; nở vật cản thêm atan(r/d) theo bán kính drone; ngoài FOV = chặn	Position mode + `CP_DIST`: PX4 tự dừng trước tường; so với model `gz_x500_lidar_2d`
5 · Planner	Histogram + odom + đích → hướng + tốc độ	Topic nội bộ `/planner/cmd`	cost = a·vật cản + b·lệch đích + c·đổi hướng; hysteresis; phát hiện kẹt sau 5 s → báo state machine sang RECOVER	Scenario runner + nhóm B trong Edge cases
6 · Safety gate	Lệnh + tuổi dữ liệu + heartbeat → lệnh an toàn	`/safety/cmd`	Depth cũ > 0.3 s hoặc node mất heartbeat → hover; vật < 0.8 m → phanh; kẹp v ≤ v_max; geofence	Cố ý tắt camera giữa chuyến, kill node, spawn vật sát mặt

Tầng	Input → Output	Message (gợi ý)	Tham số khởi đầu	Test băng
7 · Offboard cmd	Lệnh → setpoint PX4	`TrajectorySetpoint` + `OffboardControlMode`	⩾ 10 Hz; PX4 bắt buộc ⩾ 2 Hz (PX4 docs); ngừng gửi khi state machine ở ABORT	Tắt node → PX4 failsafe đúng (`COM_OF_LOSS_T`)
8 · PX4 + Gazebo	Setpoint → chuyển động + odometry	`VehicleOdometry` , `VehicleStatus`	Mặc định	ULog + PlotJuggler: setpoint so với thực tế

Tốc độ tối đa an toàn

Drone phải dừng kịp trong khoảng còn trống, tính cả quãng bay trong lúc hệ thống còn đang xử lý. Với d là khoảng cách tới vật cản, d_safe là lề an toàn, a là gia tốc phanh, t_l là độ trễ từ lúc chụp ảnh tới lúc ra lệnh:

```
v_{max} = -a\,t_l + \sqrt{(a\,t_l)^2 + 2a\,(d - d_{safe})}
```

Ví dụ: a = 2 m/s², t_l = 0.3 s, d = 4 m, d_safe = 1 m → v_max ≈ 2.9 m/s. Sang giai đoạn 4, model chậm hơn làm t_l tăng, nên công thức này tự hạ tốc độ.

Metric của vòng đánh giá

Metric	Đo gì	Mục tiêu giai đoạn 3
Success rate	Tới đích, không va chạm	⩾ 90% trên 50 world random
Min clearance	Khoảng cách nhỏ nhất tới vật cản	⩾ 0.5 m
Time-to-goal	So với bay thẳng không vật cản	⩽ 2×
Latency p95	Từ timestamp ảnh tới setpoint	⩽ 100 ms (với sim depth)
Depth AbsRel (giai đoạn 4)	Sai số model so với sim depth	Ghi lại để so, chưa đặt ngưỡng

Edge cases & cách xử lý

Có 30 edge case test được hoàn toàn trong sim. Cột Test là scenario đưa vào bộ test tự động; giai đoạn 3 chỉ tính là xong khi qua hết nhóm A–C. Quy tắc chung: **không chắc thì coi là bị chặn**, không bao giờ coi là trống.

A. Perception (tầng 1–4)

Edge case	Phát hiện	Xử lý	Test
Vật quá gần (dưới tầm tối thiểu), pixel thành NaN/inf	Tỉ lệ pixel không hợp lệ ở vùng giữa tăng vọt, frame trước đã có vật gần	Coi là vật cản sát mặt, không phải khoảng trống	Spawn tường cách mũi 0.2 m
Ngoài tầm xa (> 10 m)	Pixel không hợp lệ đồng loạt ở xa, không có vật gần trước đó	Coi là trống tới 10 m	World trống
Drone nghiêng thấy sàn/trần	Pitch/roll từ odometry	Chiếu điểm sang hệ thế giới (có extrinsics camera) rồi mới lọc độ cao	Tăng tốc gắp trên sàn phẳng: 0 vật cản giả
Vật mảnh (cột Ø 5 cm, dây)	Vật chỉ chiếm vài pixel	Percentile theo từng cột ảnh, không gộp cả bin; nở vật cản theo bán kính drone	Cột 5 cm cách 3 m
Khe hẹp hơn thân drone + lề	—	Nở mỗi vật cản thêm góc atan(r/d) nên khe tự đóng	Hai tường cách nhau 0.6 m
Camera ngừng gửi / rớt frame	Tuổi frame > 0.3 s	Safety gate cho hover	Kill node camera giữa chuyến
Timestamp lệch, quên use_sim_time	Frame cũ hơn frame trước hoặc lệch đồng hồ sim	Bỏ frame, cảnh báo; mọi node dùng sim time	Tắt use_sim_time ở 1 node, phải bị phát hiện
Hướng nằm ngoài FOV	Bin không có dữ liệu	Coi là bị chặn; muốn bay hướng đó phải quay mũi trước	Đích nằm bên hông 90°
Depth nhấp nháy giữa các frame (giai đoạn 4)	Độ lệch giữa 2 frame liên tiếp lớn	Median theo thời gian, giữ giá trị gần hơn	Phát lại bag từ model
Tường trơn, kính, thiếu sáng (giai đoạn 4)	Nhiều lỗ trong depth stereo	Lỗ ở vùng giữa = chặn, giảm tốc	World tường một màu không hoạ tiết

B. Planner (tầng 5)

Edge case	Phát hiện	Xử lý	Test
Ngõ cụt / chữ U	Không tiến gần đích hơn sau 5 s	RECOVER: xoay quét 360°, lùi, thử lại; quá 3 lần thì ABORT	World chữ U
Lắc trái phải giữa 2 hướng ngang nhau	Hướng chọn đổi chiều > 3 lần trong 2 s	Hysteresis: giữ hướng cũ trừ khi hướng mới tốt hơn rõ rệt	Cột đặt đúng giữa đường bay
Đích nằm trong vật cản / không tới được	Recovery thất bại liên tiếp	ABORT: hover rồi hạ cánh, ghi lý do	Đích đặt trong hộp kín
Đã tới đích	Khoảng cách < 0.3 m	Dừng, giữ vị trí, không bay vòng quanh	Mọi run
Vật lơ lửng hoặc vật thấp	Dải trên/dưới của histogram	Giữ độ cao cố định; dải nào bị chặn thì cả hướng đó bị chặn	Xà ngang đúng tầm bay
Vật xuất hiện đột ngột	Khoảng cách giảm nhanh hơn tốc độ bay	Safety gate phanh trước, planner tính lại sau	Spawn hộp trước mặt khi đang bay
Góc yaw nhảy ±π	—	Chuẩn hoá góc về $(-\pi, \pi]$ ở mọi phép tính	Unit test, đích nằm ngay sau lưng

C. Safety & điều khiển (tầng 0, 6–8)

Edge case	Phát hiện	Xử lý	Test
Node perception/planner crash	Mất heartbeat của node	Safety gate cho hover	Kill node
Node Safety gate hoặc offboard crash	PX4 không còn nhận setpoint	PX4 tự failsafe sau `COM_OF_LOSS_T`; kiểm tra hành vi đã cấu hình	Kill node offboard
PX4 tự thoát offboard (pin yếu, geofence, RC)	`nav_state` đổi khỏi OFFBOARD	State machine sang ABORT, ngừng gửi lệnh, không giành lại quyền	Bật failsafe pin yếu trong sim
Arm/cất cánh bị từ chối (preflight fail)	Lệnh arm trả lỗi	Thử lại có giới hạn, rồi báo lỗi	Ngắt kết nối GCS

Edge case	Phát hiện	Xử lý	Test
Odometry nhảy (EKF reset)	Vị trí đổi > 1 m trong 1 bước	Hover, reset planner	Tiêm bước nhảy giả vào odometry
Spawn sát vật cản	Kiểm tra histogram trước khi cất cánh	Không cất cánh, báo lỗi	Spawn cách tường 0.3 m
Bay nhanh hơn khả năng dừng	v > v_max theo công thức	Kẹp tốc độ	Đích xa trong world dày
Nhầm hệ trục NED/ENU	—	Hàm chuyển trục dùng chung + unit test	Unit test

D. Vòng đánh giá

Edge case	Phát hiện	Xử lý
Va chạm mà không ai biết	Contact sensor trong Gazebo, hoặc khoảng cách từ vị trí thật tới vật	Đánh dấu run fail
World random không có đường đi	Chạy A* trên lưới 2D trước khi bay	Sinh lại world, không tính vào kết quả
Sim treo / crash	Watchdog theo giờ thật	Kill, ghi "lỗi hạ tầng", không tính vào success rate
Real-time factor tụt khi máy nặng	Đọc RTF của Gazebo	Mọi timeout tính bằng sim time
Drone lật hoặc bay ra ngoài world	Vị trí ngoài biên hoặc nghiêng > 60°	Fail ngay, dừng run

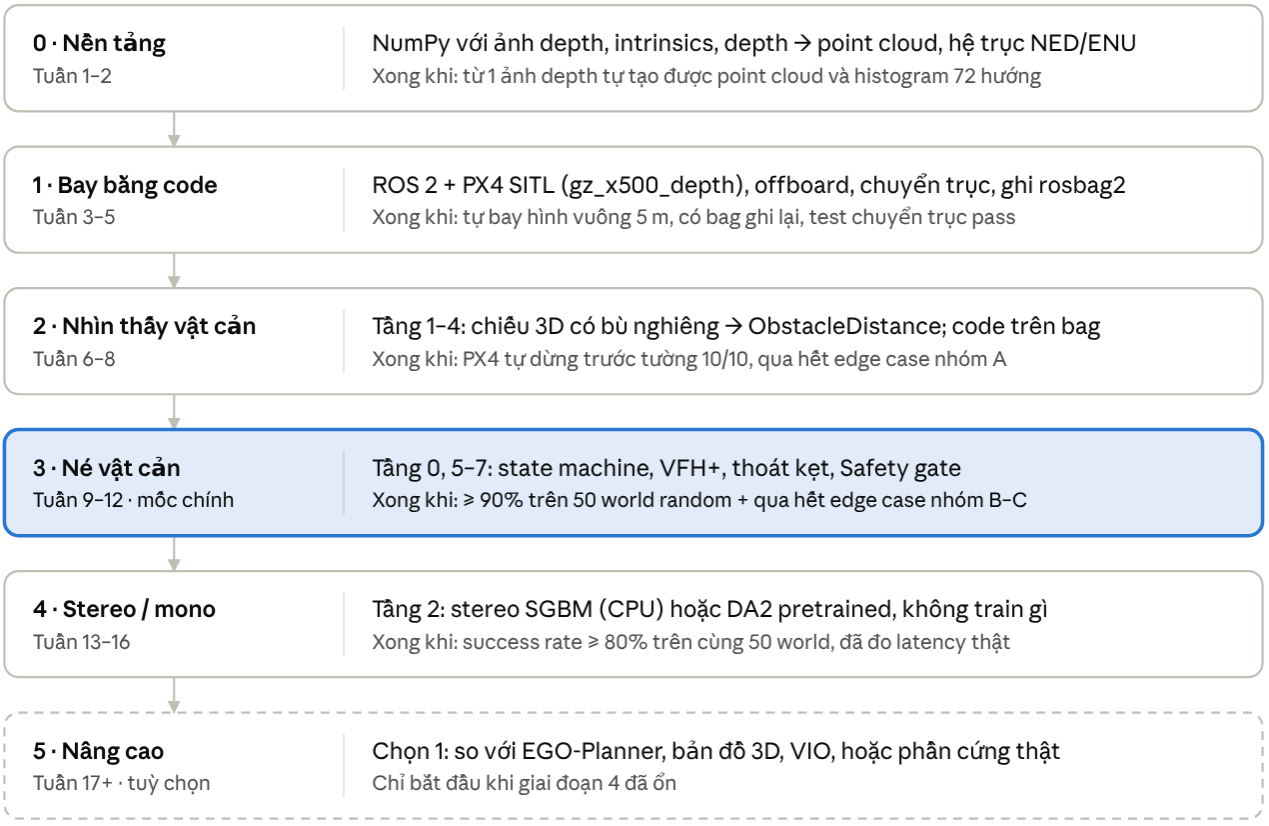
Tech stack đề xuất

Chọn combo PX4 + Gazebo Harmonic + ROS 2 Jazzy vì đây là bộ được hỗ trợ chính thức và có Docker image dựng sẵn, đỡ mất cả tuần cài đặt.

Thành phần	Chọn	Vì sao / ghi chú
Hệ điều hành	Ubuntu 24.04 (native hoặc dual boot)	WSL2 chạy được nhưng GUI Gazebo hay lỗi, nên có Ubuntu thật
Autopilot	PX4 SITL	Phần mềm bay giống hệt trên Pixhawk thật, sau này lên phần cứng ít phải sửa
Simulator	Gazebo Harmonic	Có sẵn model gz_x500_depth (drone X500 + depth camera kiểu OAK-D). AirSim gốc đã bị Microsoft bỏ, không nên bắt đầu từ đó
Middleware	ROS 2 Jazzy + Micro XRCE-DDS	Cầu nối giữa PX4, camera và code của bạn
Môi trường nhanh	Docker image `px4io/px4-sitl-gazebo-ros2` (Docker Hub)	Gói sẵn Ubuntu 24.04, ROS 2 Jazzy, PX4 SITL, Gazebo Harmonic, không cần tự build PX4
Ngôn ngữ	Python (rclpy)	Nhanh để thử; phần nặng có thể viết lại bằng C++ sau
Xử lý ảnh	OpenCV, NumPy, cv_bridge	Nền tảng bắt buộc phải nắm
Ước lượng độ sâu (giai đoạn 4)	Depth Anything V2 Small (Hugging Face)	Bản Small 24.8M tham số, Apache 2.0, có bản metric. So thêm với Depth Anything 3 (ra 11/2025, có DA3METRIC-LARGE) khi chạy offline
Detect vật thể (tuỳ chọn)	YOLO (Ultralytics)	Chỉ cần nếu muốn biết vật cản *là gì* (người, cây…)
Phần cứng sau này	Pixhawk + Jetson Orin Nano + camera stereo/depth	Chưa mua, chỉ để định hướng

Lộ trình theo giai đoạn

Mốc chính: né vật cản tự động ở tuần 12, đo bằng 50 world random



lộ trình · 6 giai đoạn, mỗi giai đoạn có tiêu chí xong

Chỉ sang giai đoạn mới khi đạt dòng "Xong khi" của giai đoạn trước; số tuần chỉ là ước lượng với 8–10 giờ/tuần.

Các hướng tiếp cận tránh vật cản

Hướng 1 chỉ dùng vài ngày để debug cho dễ nhìn; mục tiêu thật của giai đoạn 3 là hướng 2 (VFH+ trên histogram 72 hướng), vì hướng 1 quá thô (xem Phản biện). Hướng 3–4 để dành cho giai đoạn 5.

#	Hướng	Cách làm ngắn gọn	Độ khó	Điểm yếu
1	Chia vùng ảnh depth (reactive)	Chia ảnh depth thành trái/giữa/phải, lấy khoảng cách gần nhất mỗi vùng, nếu giữa < ngưỡng thì rẽ sang vùng thoáng hơn	Dễ	Dễ kẹt trong ngõ cụt, không nhớ vật đã đi qua

#	Hướng	Cách làm ngắn gọn	Độ khó	Điểm yếu
2	Potential field / VFH	Biến depth thành histogram theo hướng, kết hợp lực hút về goal và lực đẩy từ vật cản	Trung bình	Vẫn có local minimum, phải tune hệ số
3	Bản đồ 3D + planner	Dựng occupancy map (OctoMap/voxel) từ depth, chạy A* hoặc RRT* trên map	Khó	Cần vị trí drone chính xác, tốn tài nguyên
4	Học máy end-to-end / RL	Train model từ ảnh ra lệnh bay, thường trong simulator	Rất khó	Cần nhiều data/GPU, khó debug, dễ fail khi qua thế giới thật

Nguồn độ sâu, xếp theo thứ tự nên thử: depth camera giả lập (khoảng cách gần như hoàn hảo) → stereo camera + OpenCV StereoSGBM → camera đơn + Depth Anything V2 (độ sâu tương đối, phải xử lý tỉ lệ hoặc dùng bản metric).

Phản biện: plan cũ có thể fail ở đâu

Nghĩ ngược lại ("làm sao để project này chắc chắn fail?") thì ra 12 lỗ hổng. 3 lỗ đầu nghiêm trọng nhất vì nó làm hỏng cả kiến trúc; mọi cách sửa bên dưới đã được đưa vào Pipeline và Lộ trình.

#	Cách để fail	Vì sao fail	Sửa thành
1	Làm theo tutorial PX4-Avoidance	Repo đã bị archive từ 08/2024, và PX4 1.16 đã gỡ luôn interface avoidance (thảo luận PX4)	Tự viết planner qua ROS 2; sau đó so với EGO-Planner, planner đã có người chạy chung với PX4 + ROS 2 Jazzy (paper)
2	Tin rằng PX4 Collision Prevention sẽ chặn va chạm khi bay offboard	Collision Prevention chỉ chạy ở Position mode (PX4 docs), bật offboard là mất lớp bảo vệ	Tự viết Safety gate. Chỉ dùng Collision Prevention ở giai đoạn 2 để kiểm tra perception
3	Lấy khoảng cách nhỏ nhất trên cả ảnh depth	Sàn, trần và vài pixel nhiễu làm drone lúc nào cũng thấy vật cản	Cắt ROI theo độ cao, bỏ điểm sàn/trần, lấy percentile 5% thay vì min, lọc median theo thời gian

#	Cách để fail	Vì sao fail	Sửa thành
4	Chỉ chia 3 vùng trái/giữa/phải	Quá thô: kẹt giữa 2 vật, lọt khe, bỏ sót vật mảnh	Histogram 72 hướng × 5° (cùng format ObstacleDistance của PX4), planner kiểu VFH+
5	Không tính độ trễ và quãng phanh	Bay nhanh hơn khả năng dừng, nhất là khi depth từ model chậm	Giới hạn vận tốc theo khoảng cách còn trống và độ trễ đo được (công thức ở Pipeline)
6	Camera chỉ nhìn phía trước nhưng drone bay ngang/lùi	Lao vào vật nằm ngoài tầm nhìn	Luôn quay mũi (yaw) theo hướng bay; cấm bay vào hướng không có dữ liệu
7	Đếm "18/20 lần" bằng tay	Kết quả không lặp lại được, không so được giữa các phiên bản	Scenario runner: world random theo seed, chạy headless hàng loạt, tự tính metric
8	Phát triển perception bằng cách bay sim liên tục	Mỗi lần thử mất vài phút, khó tái hiện bug	Ghi rosbag2 một lần, phát lại để code perception offline
9	Depth giả lập hoàn hảo	Code "quá khớp" với dữ liệu sạch, gặp nhiễu thật là vỡ	Tự thêm nhiễu Gaussian, lỗ hổng (dropout), giới hạn tầm 0.3–10 m trong node
10	Dùng depth tương đối từ model rồi so với ngưỡng mét	Độ sâu tương đối không có đơn vị, ngưỡng 1.5 m vô nghĩa	Dùng model metric (DA2 metric hoặc DA3METRIC), đo sai số trên bag trước khi cho bay
11	Nhầm hệ trục NED/FRD (PX4) với ENU/FLU (ROS)	Drone bay ngược hướng hoặc xoáy vòng, bug kinh điển	Viết hàm chuyển trục + unit test ngay giai đoạn 1
12	Quen với odometry hoàn hảo của sim	Bay thật trong nhà không có GPS, vị trí trôi	Giai đoạn 5 test với odometry có nhiễu, chuẩn bị VIO trước khi lên phần cứng

Ngoài ra, giai đoạn 0 cũ (lọc màu, contour webcam) lệch đề: đã thay bằng xử lý mảng depth bằng NumPy và mô hình camera.

Rủi ro & cách giảm

Rủi ro lớn nhất với người mới là bị chôn ở khâu cài đặt và bỏ cuộc trước khi viết dòng CV đầu tiên.

Rủi ro	Dấu hiệu	Cách giảm
Kẹt cài đặt PX4/ROS 2/Gazebo	Mất hơn 1 tuần vẫn chưa thấy drone cất cánh	Dùng Docker image dựng sẵn; chỉ làm theo docs bản Jazzy/Harmonic, bỏ qua tutorial cũ dùng `ign` hay Gazebo Classic
Máy yếu	Gazebo giật, FPS camera thấp	Chạy headless, giảm độ phân giải camera, world đơn giản
Ôm quá nhiều thứ cùng lúc	Học ROS 2, CV, deep learning song song, không cái nào xong	Bám lộ trình, chỉ qua giai đoạn mới khi đạt tiêu chí gate
Drone bay loạn khi offboard	Mất kiểm soát, rơi trong sim	Gửi setpoint đều đặn, giới hạn vận tốc, luôn có nút chuyển về Hold/Land
Depth từ model sai tỉ lệ	Né quá sớm hoặc quá trễ	So sánh với depth camera giả lập (ground truth), dùng bản metric hoặc ngưỡng tương đối
Sim tốt nhưng thật fail (sim-to-real gap)	Chỉ lộ ra khi lên phần cứng	Thêm nhiễu/mờ/ánh sáng khác vào sim; test với video quay thật trước khi bay

Tài nguyên học

Nếu chỉ đọc 5 thứ: ví dụ PX4 Offboard, bài báo VFH+, repo mogi-ros Gazebo sensors, khoá Scaramuzza (phần image formation + stereo), trang PX4 Gazebo vehicles. Ưu tiên docs đúng phiên bản Jazzy/Harmonic; tutorial dùng ROS 1, Gazebo Classic hoặc PX4-Avoidance là đã cũ.

Giai đoạn	Tài liệu	Dùng để làm gì
0	OpenCV-Python Tutorials	Xử lý ảnh cơ bản, làm việc với mảng NumPy

Giai đoạn	Tài liệu	Dùng để làm gì
0	Vision Algorithms for Mobile Robotics – Scaramuzza (UZH/ETH)	Slide + video: image formation, hiệu chỉnh camera, stereo, visual odometry. Đúng hướng robot/drone
0–4	Szeliski – Computer Vision: Algorithms and Applications, 2nd ed.	PDF miễn phí cho cá nhân. Chỉ đọc chương mô hình camera và stereo
1	ROS 2 Jazzy Tutorials	Node, topic, launch, parameter
1	PX4 ROS 2 Offboard Control example	Code mẫu trong px4_ros_com: gửi OffboardControlMode + TrajectorySetpoint, arm, cất cánh 5 m. Ví dụ test trên ROS 2 Foxy + PX4 v1.14, cần chỉnh cho Jazzy
1	PX4 User Guide – ROS 2	Cài XRCE-DDS, px4_msgs; đọc kỹ phần hệ trục NED/ENU vì PX4 không tự chuyển
1	PX4 Offboard mode	Luật heartbeat ⩾ 2 Hz, các kiểu setpoint được hỗ trợ
1	PX4 Gazebo vehicles	Lệnh chạy gz_x500_depth, gz_x500_lidar_2d
1	px4io/px4-sitl-gazebo-ros2	Môi trường dựng sẵn, khỏi cài tay
1–2	Gazebo Harmonic docs	World SDF, sensor, real-time factor
2	mogi-ros/week-5-6-gazebo-sensors	Bài tập Gazebo Harmonic + ROS 2: thêm RGBD camera, bridge, đọc depth bằng OpenCV, point cloud
2	ros_gz_bridge	Bảng ánh xạ kiểu message Gazebo ↔ ROS 2
2	PX4 Collision Prevention	Test perception qua ObstacleDistance ở Position mode
2, 4	Towards Real-Time Monocular Depth Estimation for Robotics: A Survey	Phần obstacle avoidance tóm tắt cách UAV dùng depth map, có cả quãng phanh
3	VFH+ (Ulrich & Borenstein, ICRA 1998)	Thuật toán cho tầng 5; ngắn, đọc kỹ 4 bước giảm dữ liệu

Giai đoạn	Tài liệu	Dùng để làm gì
3	VFH* (Ulrich & Borenstein, ICRA 2000)	VFH+ cộng tìm kiếm A* nhìn trước, xử lý tình huống kẹt của planner thuần local. Tham khảo cho thoát kẹt
3	PlotJuggler	Mở ULog PX4 và rosbag ROS 2, vẽ setpoint so với thực tế
3	Foxglove + PX4 ULog	Xem log PX4, thay cho RViz khi debug
4	Depth Anything V2 Small	Model pretrained nhẹ, chỉ inference
4	Depth Anything 3	So sánh offline trên Colab/Kaggle, có bản metric
4	StereoPi: OpenCV depth map tutorial	Quy trình stereo đầy đủ: hiệu chỉnh → tune depth map → chạy real-time, trên Raspberry Pi
4	Image-Based Obstacle Detection Methods for Unmanned Vehicles: A Review	Phân loại cách dùng camera đơn vs stereo, có mục disparity histogram cho drone
5	EGO-Planner	Planner để so với bản tự viết; bản ROS 2 nằm ở nhánh ros2_version của ego-planner-swarm
5	Droneulator (EGO-Planner + PX4 + ROS 2 Jazzy)	Ví dụ nối EGO-Planner vào PX4 qua XRCE-DDS
5	OpenVINS	VIO cho bay trong nhà không GPS, có hỗ trợ ROS 2
5	Vision-Based Learning for Drones: A Survey	Tổng quan nếu sau này muốn đi hướng học máy

Lưu ý license nếu định public code: EGO-Planner và OpenVINS là GPL-3.0; Depth Anything V2 Small và Depth Anything 3 là Apache 2.0. Bối cảnh PX4 gỡ interface avoidance: PX4 Sync 01/2025.

Câu hỏi mở để thảo luận

- [ ] Máy bạn cấu hình gì (CPU, RAM, có GPU NVIDIA không), đang chạy Windows hay Ubuntu?

- [ ] Muốn né trong nhà (tường, cửa, bàn ghế) hay ngoài trời (cây, cột, dây điện)? Dây điện mảnh là bài toán khó hơn nhiều.
- [ ] Chọn depth camera giả lập làm bước đầu, hay muốn đi thẳng camera đơn + model ngay từ đầu để học CV nhiều hơn?
- [ ] Cuối cùng có muốn lên drone thật không? Nếu có, ngân sách tầm bao nhiêu để chọn Raspberry Pi hay Jetson?
- [ ] Có muốn biến project thành portfolio (GitHub + video demo + bài viết) không? Nếu có, nên ghi log mỗi giai đoạn ngay từ đầu.